

Phase 2 :Security Assessment Report

TransactiWar

Team-12

March 24, 2026

1 Introduction

1.1 Web Application Functionality

Web Application that we made is a PHP-MySQL web application for wallet-style user transactions. Core features include:

- User registration, login, and logout.
- Dashboard with current balance and recent transaction summary.
- Money transfer between users with transfer comments.
- Transaction history view.
- User search and profile viewing.
- Editable own profile with optional profile image upload.

1.2 Core Security Mechanisms Initially Implemented

The application includes the following baseline controls:

- Parameterized PDO queries across authentication, profile, transfer, search, logs, and rate-limiting flows.
- CSRF token generation and verification for state-changing forms (login, registration, transfer, profile update).
- Session hardening: strict mode, HttpOnly cookies, SameSite=Lax, session ID regeneration on login/registration, and logout-time session destruction.
- Password hashing and verification using `password_hash()` and `password_verify()`.
- Input validation for usernames, emails, passwords, and transfer amounts.
- Brute-force defense with login rate limiting by email/IP over a 15-minute window.
- Transaction safety through DB transactions and `SELECT ... FOR UPDATE` row locks.

- File-upload safeguards (extension/MIME checks, image content checks, size limits) and protected image serving for authenticated users only.
- Activity logging to both a file log and database logs table.
- Security headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy).

2 War Game Attack Analysis

2.1 Successful Attacks Conducted by Our Team

2.1.1 Negative Amount Handling in Money Transfer

Discovered by: Aditya Waghmare

Target: Team 19

Severity: High

Description: The money transfer functionality does not properly validate the `amount` parameter. By intercepting the request and modifying the amount to a negative value, the application processes the request instead of rejecting it. This results in unintended backend behavior and exposes an internal error message.

Steps to Reproduce:

1. Login to the application using valid credentials.
2. Navigate to the **Transfer Money** page.
3. Intercept the request using Burp Suite.
4. Modify:

```
amount = -1000
```

5. Forward the request.
6. Observe the error message.

Impact:

- Improper validation of transaction values.
- Backend executes inconsistent database operations.
- Internal error message disclosure.
- Could potentially be abused to manipulate account balances if additional checks are missing.

Proof:

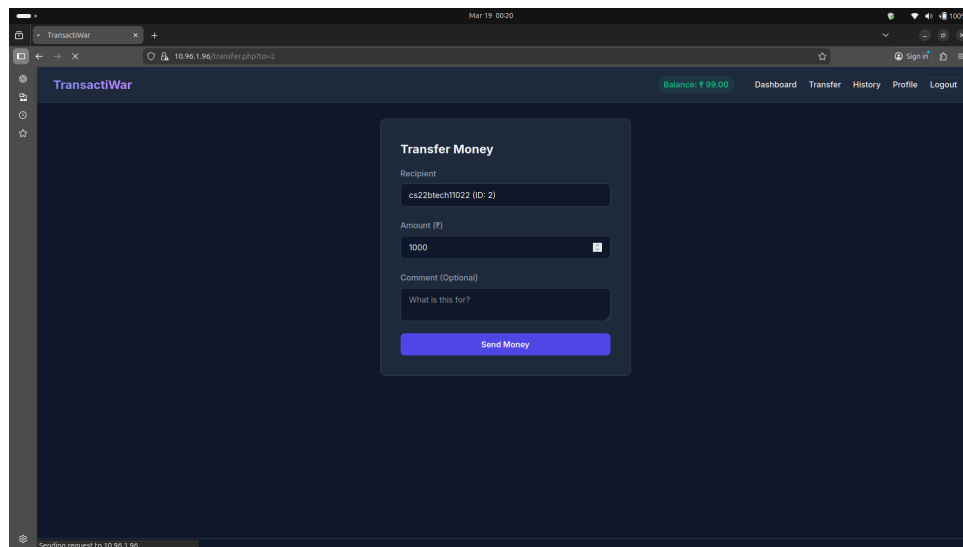


Figure 1: Transfer page before modification

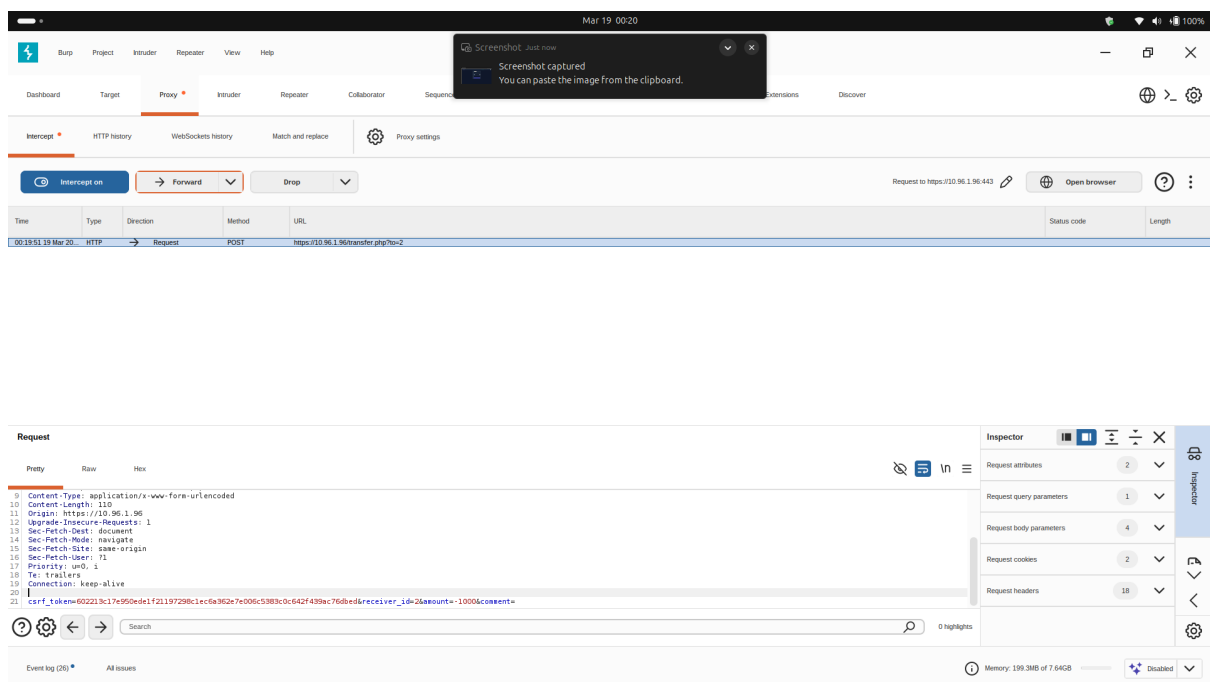


Figure 2: Modified request with negative amount

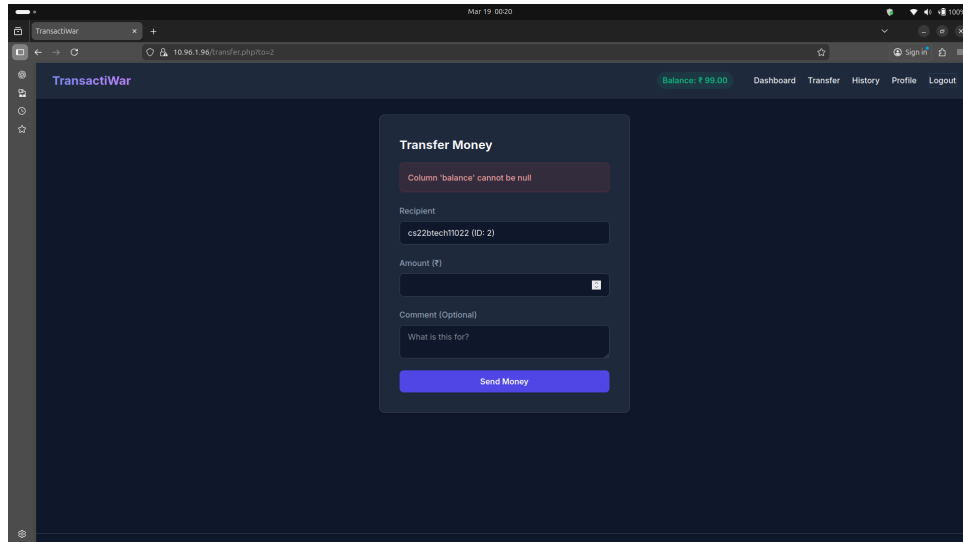


Figure 3: Database error message exposed

2.1.2 CSRF Vulnerability in Profile Update

Discovered by: Aditya Waghmare

Target: Team 11

Severity: Medium

Description: The profile update endpoint does not implement CSRF protection. A malicious page can force a logged-in user to update their profile without consent.

Steps to Reproduce:

1. Login to the application.
2. Intercept profile update request.
3. Observe absence of CSRF token.
4. Create malicious page:

```
<html>
<body onload="document.forms[0].submit()">

<form action="https://10.96.0.60/profile/edit.php" method="POST" enctype="multipart/form-data">
  <input type="hidden" name="email" value="attacker@evil.com">
  <input type="hidden" name="biography" value="CSRF_PROFILE_HACK">
</form>

</body>
</html>
```

5. Open while logged in.

Impact:

- Unauthorized profile modification.
- Can be triggered without user interaction.

Proof:

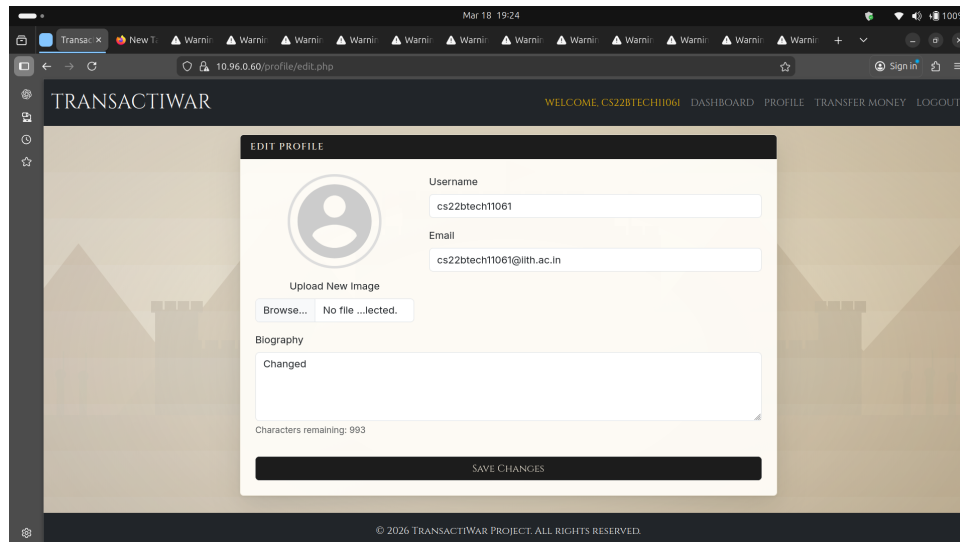


Figure 4: Profile before attack

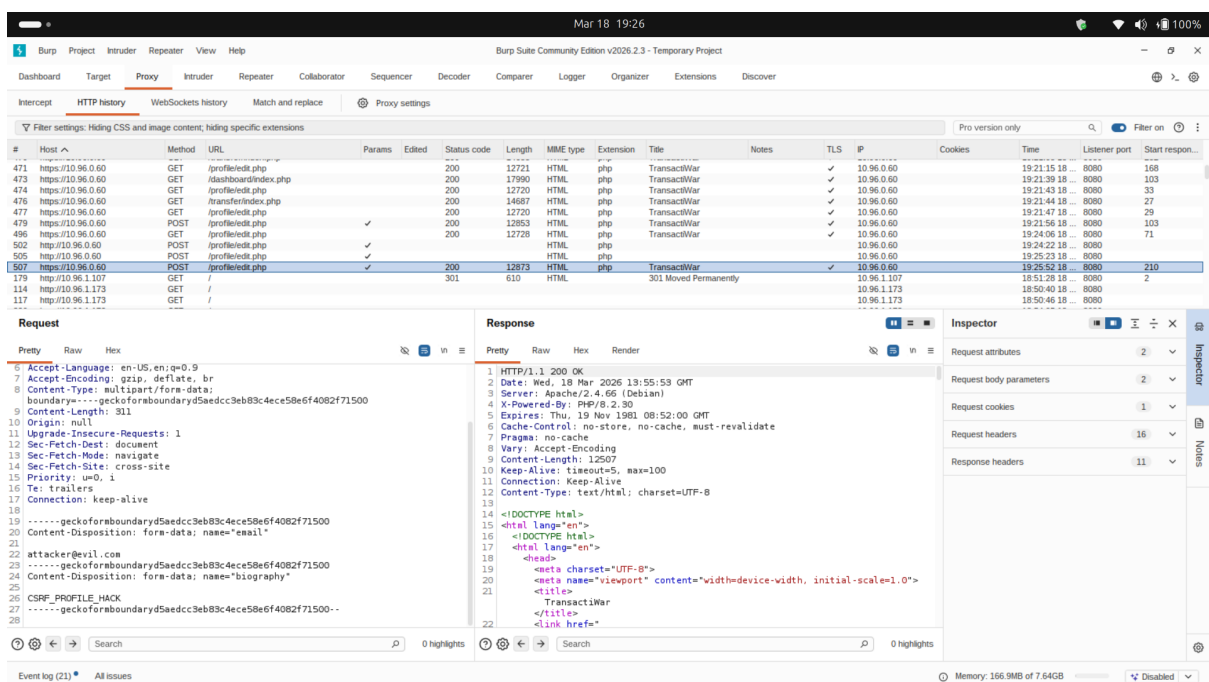


Figure 5: No CSRF token in request

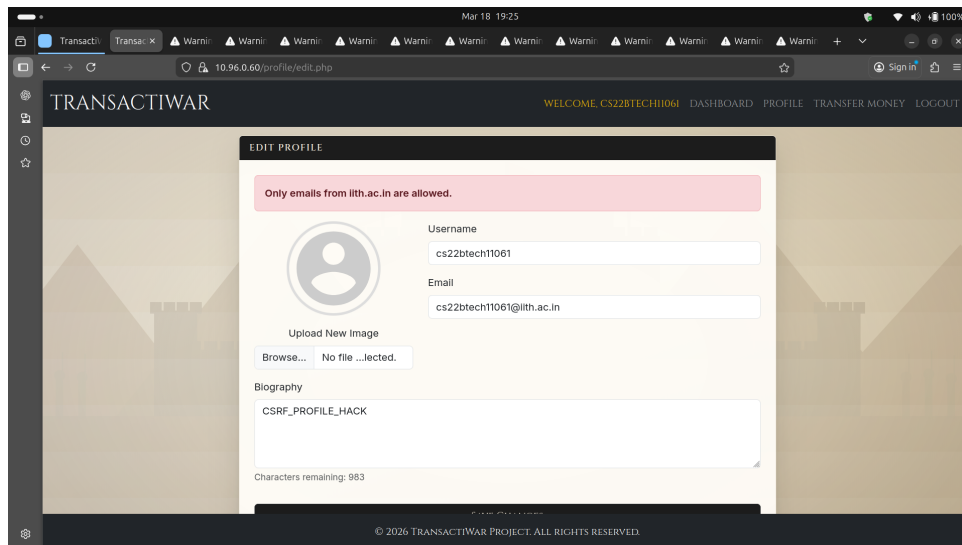


Figure 6: Profile modified via CSRF

2.1.3 CSRF in Money Transfer Endpoint

Discovered by: Aditya Waghmare

Target: Team 11

Severity: High

Description: The transfer endpoint processes requests without CSRF validation, allowing unauthorized fund transfers.

Steps to Reproduce:

1. Login as victim.
2. Intercept transfer request.
3. Observe no CSRF token.
4. Create malicious HTML:

```
<html>
<body onload="document.forms[0].submit()">

<form action="https://10.96.0.60/transfer/index.php" method="POST">
  <input type="hidden" name="receiver_id" value="WAR-60A5E6ED">
  <input type="hidden" name="amount" value="5">
  <input type="hidden" name="comment" value="CSRF_ATTACK">
</form>

</body>
</html>
```

5. Open while logged in.

Impact:

- Unauthorized transfer of funds.
- Direct financial loss.
- Can be combined with phishing attacks to perform large-scale unauthorized transactions.

Proof:

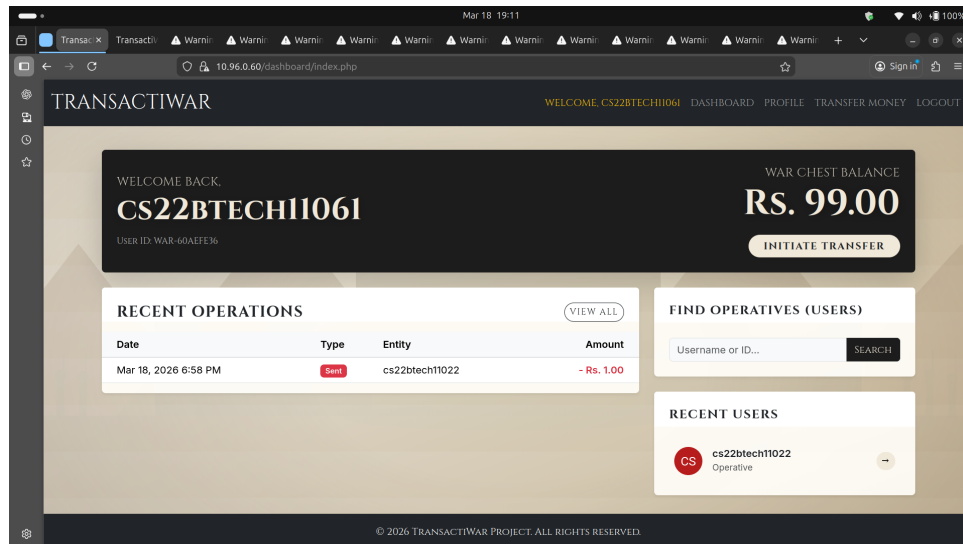


Figure 7: Balance before attack

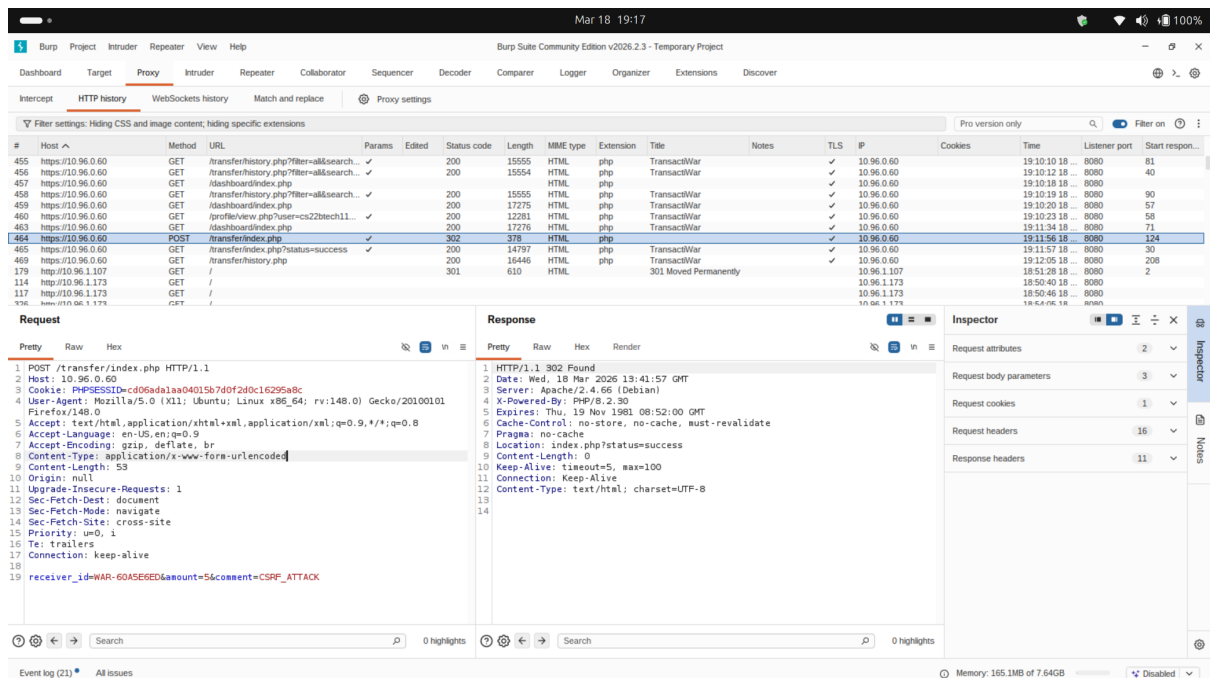


Figure 8: Transfer request without CSRF protection

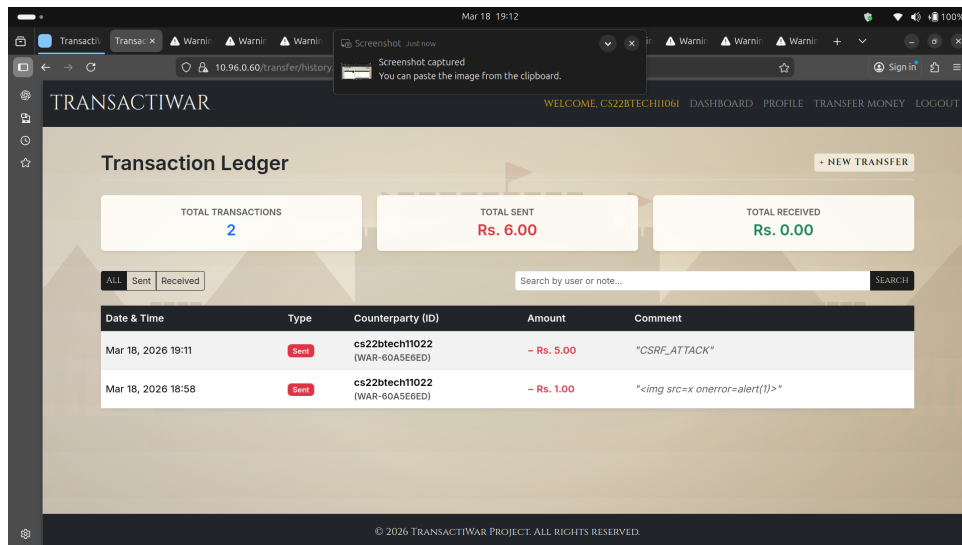


Figure 9: Unauthorized transaction executed

2.1.4 Application Unavailability During Assessment

Discovered by: Aditya Waghmare

Target: Team 14

Severity: High

Description: During the assessment phase, the target application was found to be completely inaccessible. Both HTTP and HTTPS endpoints were not responding, indicating that the service was down at the time of testing.

Steps to Reproduce:

1. Attempt to access the application at:

```
http://10.96.1.160
```

```
https://10.96.1.160
```
2. Observe that the server does not respond.
3. The browser displays a timeout error.

Impact:

- Application was unavailable for testing.
- Prevented further security assessment of the system.
- Indicates lack of reliability or deployment stability.

Note: This issue was observed passively during testing. No active Denial of Service attack was performed.

Proof:

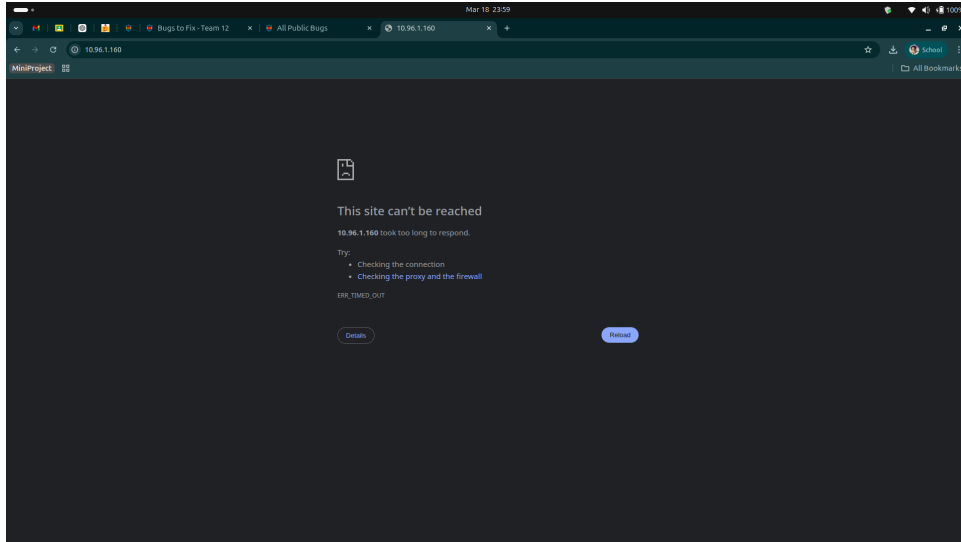


Figure 10: Application unreachable during testing

3 Attacks We Tested on Our Application and Defenses

We tested multiple attack vectors on our system as part of internal security testing. The following sections describe the attacks we performed in a controlled environment and the security measures we added. These measures were implemented to fix vulnerabilities found during testing and to prevent unauthorized access.

3.1 Summary Table

Attack Type	Current Status	Primary Defense
SQL injection	Mitigated	Prepared statements everywhere, PDO native prepares enabled
Stored/reflected XSS	Mitigated	Output escaping with <code>htmlspecialchars()</code> in views + CSP
CSRF on state-changing actions	Mitigated	CSRF token generation and <code>hash_equals()</code> validation
Session fixation/hijacking	Mitigated	Strict mode, HttpOnly, SameSite, login-time regeneration
Password cracking/Guessing	Mitigated	Password hashing with <code>password_hash()</code>
Brute-force login attempts	Mitigated	15-minute rate limiting by email/IP
File upload to RCE path	Mitigated	MIME/extension/image checks + upload outside web root
Path traversal / local file read	Mitigated	Filename allowlist + <code>realpath()</code> prefix enforcement

3.2 SQL Injection

Attack: Inject SQL through login, register, search, transfer, profile, or logging inputs.

Defense present:

- Controllers and middleware use parameterized queries with placeholders for all observed SQL operations.

3.3 Cross-Site Scripting (XSS)

Attack: Inject script in bio/comment/username or reflected query output, then execute in browser.

Defense present:

- Views consistently escape dynamic data via `htmlspecialchars()`.
- Numeric outputs are cast to integers for IDs.
- Content Security Policy and additional security headers are set in `secureSessionStart()`.

3.4 Cross-Site Request Forgery (CSRF)

Attack: Force authenticated users to submit unwanted actions (transfer, profile update, login/register post abuse).

Defense present:

- 32-byte random CSRF token generated and stored in session.
- Validation uses `hash_equals()` and invalidates token after successful validation.
- Hidden token field used in login, register, transfer, and profile forms.

3.5 Session Attacks (Fixation/Hijacking)

Possible Attack: Pre-set a known session ID, steal cookie, or reuse stale session after logout.

Defense present:

- reject unknown IDs.
- Cookies configured as `HttpOnly`, `SameSite=Lax`, and `Secure` when HTTPS is detected.
- Session ID regeneration on successful login/registration.
- Logout destroys session data and expires cookie.

3.6 Credential Theft and Password Cracking

Possible Attack: Weak password abuse.

Defense present:

- Passwords are hashed using `password_hash()` and verified with `password_verify()`.
- Registration enforces complexity via regex (upper, lower, digit, special, length).

3.7 Brute Force and Credential Stuffing

Possible attack: Repeated login attempts across email/IP combinations.

Defense present:

- Rate limiting in `login_attempts`: lockout after 5 failed attempts in 15 minutes by email/IP.

3.8 Malicious File Upload and File Handling

Attack: Upload script file, bypass content checks, or fetch arbitrary server files.

Defense present:

- Upload validation checks upload source, extension allowlist, size boundaries, MIME, and image content.

3.9 Access Control and IDOR

Attack: Access another user's protected data or perform unauthorized actions.

Defense present:

- Protected controllers require authenticated session.
- Profile updates are scoped to session `user_id`.
- View-profile logic redirects when user attempts to view their own profile through other-profile endpoint.
- Self-transfer is blocked.

4 Attacks Conducted by Other Teams

All attacks conducted against our web application were unsuccessful in obtaining unauthorised access to private data, including transaction records and log data.

4.1 Attack Summary Table

#	Attack Type	Severity	Description	Status	Reason Invalid / Mitigation
1	Session Hijacking via Reuse of Session ID	HIGH	Reported by Team 4 (Yadav Rohit Rajesh).The attacker claimed that session IDs were being reused, potentially allowing session hijacking after a user logs in.	Invalid	We generate a new session ID every time a user logs in, so old IDs cannot be reused. We tried this attack and could not reproduce it. Most likely, there was an issue in the attacker's testing setup.
2	Website Unavailability (Site Down)	MEDIUM	Reported by Team 11 (Khawaja Abdul Samad).The attacker claimed that our website was down and unavailable.	Invalid	Our website was working when we checked it from multiple devices. We could not reproduce the downtime issue. It looks like a network/connectivity problem on the attacker's side, not an app issue.

#	Attack Type	Severity	Description	Status	Reason Invalid / Mitigation
3	SQL Injection (SQLi) Authentication Bypass on Login Form	HIGH	Reported by Team 6 (Vinay Kumar Kadari) The attacker claimed that the login form was vulnerable to SQL Injection, potentially allowing authentication bypass.	Invalid	Our app uses parameterized queries, so SQL injection on login is blocked. The screenshot shared did not show a successful bypass or clear steps. We tested it ourselves and could not reproduce the attack.
4	Information Disclosure – PHP Version Leak	MEDIUM	Reported by Team 1 (Mayank Mishra)The attacker claimed that the PHP version was being leaked, potentially exposing server information.	Invalid	The report only showed that the PHP version was visible. No real exploit or data leak was shown. So this is just an informational point, not a valid security bug.
5	Server-side Transfer Amount Validation Bypass via Scientific Notation	MEDIUM	Reported by Team 10 (Divyansh Sevta)The attacker claimed that the transfer amount field accepted scientific notation (e.g., 1e1), bypassing server-side validation.	Invalid	Even though scientific notation input was accepted, no actual damage was shown. No wrong transfer or balance manipulation happened. Without a working exploit, this is not a valid vulnerability.
6	Missing Critical Security Headers (CSP, X-Frame-Options, HSTS, etc.)	MEDIUM	Reported by Team 1 (Rajput Digvijaysing Bhatesing).The attacker claimed that critical security headers were missing from our application’s responses.	Invalid	The report said some headers were missing, but no practical attack or impact was demonstrated. Just listing missing headers is not enough to count as a valid bug.
7	Transaction Replay Attack	HIGH	Reported by Team 7 (Gaddam Hrushikesh Raj Sai)The attacker claimed that transactions could be replayed to manipulate the system.	Invalid	No strong proof of a replay exploit was provided. The screenshot did not show a real working attack. We also have replay-prevention checks, and we could not reproduce this issue.
8	No Rate Limiting on Login – Brute Force Attack Possible	MEDIUM	Reported by Team 2 (Shah Jainam Rajeshkumar) The attacker claimed that no rate limiting existed on the login page, making brute force attacks possible.	Invalid	Our login has rate limiting after 5 failed attempts. The attacker reported lockout after 3 tries, which suggests a testing mistake. No account was broken into and no successful brute-force proof was shown.
9	UNION-Based SQL Injection on Search Endpoint	MEDIUM	Reported by Team 14 (Jay Niteshbhai Vekariya)The attacker claimed that a UNION-based SQL Injection on the search endpoint leaked the database version.	Invalid	No proper proof or reproducible steps were provided for this claim. Our search flow uses parameterized queries, which block SQL injection. We could not reproduce the reported issue.
10	PHP and Apache Version Disclosed in HTTP Headers	LOW	Reported by Team 4 (Rajput Digvijaysing Bhatesing)The attacker claimed that PHP and Apache version details were visible in HTTP response headers.	Invalid	Showing software versions in headers alone is not a vulnerability. No exploit or real impact was demonstrated. This is only an informational observation, not a valid bug.
11	HTTPS Downgrade Attack	HIGH	Reported by Karthikeyan N during TransactiWar Phase 2. The attacker claimed that the application was vulnerable to an HTTPS downgrade attack.	Invalid	This issue is not specific to our application. All teams used professor-provided VMs with locally signed certificates, so this was a common environment limitation. It should not be counted as a bug in our app.

5 Contributions

- Balaji Nandure: Money transfer module, frontend development, and general testing.
- Gadekar S.B.: Authentication, database integration, CSRF token implementation, SQL injection testing, and transaction history.
- Soham Pawar: Rate limiting, database integration, rate-limiting tests, and frontend development.

- Aditya Waghmare: Profile page, users page, file inclusion testing, and frontend development.
- Siddhant Godbole: Transaction workflows, frontend development, XSS testing, and CSRF testing.

6 Lessons Learned & Future Security Enhancements

6.1 How the Application Could Be Redesigned for Stronger Security

To improve the current design, we would do the following:

- **Secrets management:** Remove hardcoded fallback credentials and require environment-only secrets (or Docker/Kubernetes secrets).
- **Authorization model:** Introduce policy-based authorization checks at service/controller boundaries for each action, not only session-presence checks.
- **CSRF consistency:** Use one-time tokens or rotating token-per-form/session strategies with explicit token lifecycle handling.
- **Safer error handling:** Replace raw exception messages shown to end users (e.g., transfer errors) with generic user-safe messages; log detailed context server-side.
- **File security hardening:** Align upload and serve-image validation rules (remove GIF from serve whitelist unless truly supported), add antivirus scanning, and add content-disposition policy.
- **Secure transport:** Enforce HTTPS-only deployment with HSTS in production and strict TLS configuration.
- **Operational security:** Introduce centralized logging/alerting, audit trails with immutable retention, and anomaly detection for suspicious transfer patterns.

7 ANTI-PLAGIARISM STATEMENT

We certify that this assignment/report is our own work, based on our personal study and/or research, and that we have acknowledged all material and sources used in its preparation, whether books, articles, packages, datasets, reports, lecture notes, or any other document, electronic or personal communication. We also certify that this assignment/report has not previously been submitted for assessment/project in any other course lab, except where specific permission has been granted from all course instructors involved, or at any other time in this course, and that we have not copied in part or whole or otherwise plagiarized the work of other students and/or persons. We pledge to uphold the principles of honesty and responsibility at CSE@IITH. In addition, we understand our responsibility to report honor violations by other students if we become aware of them.

Names:

Siddhant Godbole, Aditya Waghmare, Soham Pawar, Gadekar S.B., Balaji Nandure

CS22BTECH11054, CS22BTECH11061, CS22BTECH11055, CS22BTECH11022
CS25MTECH11007

DATE : 24 MARCH 2026

Signature: S.G., A.W., S.R.P., S.G., B.N.